Mutex

Mutex is a portmanteau for 'Mutual Exclusion'. Mutex are commonly found in all kinds of operating systems and one of the methods for synchronising two tasks. Mutexes follow several rules.

1 Mutexes are system globals

2 They can be owned by only one process at a time

3 If a mutex is already all allocated, then the function which requests the mutex for a task and by proxy the task will block until the mutex is available

Spin locks

Spin locks are the quick synchronisation mechanisms. As the name suggest a task spins in a loop until the lock opens or the condition for the lock is false. This lock is provided by the CPU so it is only possible to use it when the CPU supports the test and set idiom or gives exclusive access to a thread or a task by either masking interrupts or locking the bus.

Memory management

In their most basic form all the tasks running on a compiler will be set of instructions to the processor, now the processor will only execute the code that resides either in it's cache or it's RAM. Multiple tasks share the same memory, even the OS resides in the same memory. So the operating system kernel must come up with a way where a task's code remains independent of other tasks and also manage it's own code and make sure that no task code can manage this. These are the functions that are the responsibilities of the memory management module of the operating system.

In most systems, physical memory is composed of two-dimensional arrays made up of cells each addressed by a row and column number. Each cell stores 1 bit. On the other hand the OS treats the memory as a one-dimensional array known as the memory map. The conversion between the logical and the physical address is carried out by the MMU on board or the operating systems.

Different OSes handled the logical memory differently, but as a rule of thumb the kernel runs it's own code in a separate memory space and the task code in it's own memory space. It's responsible for managing both it's own memory and memory of the user tasks. In fact, most OSes run in either: kernel mode or user mode, depending on the routine being executed. Kernel routines and functions run in kernel mode. While the middleware and application software runs in the user mode. If user level code wants to access something in the kernel, they must do so via system calls, theses system calls act